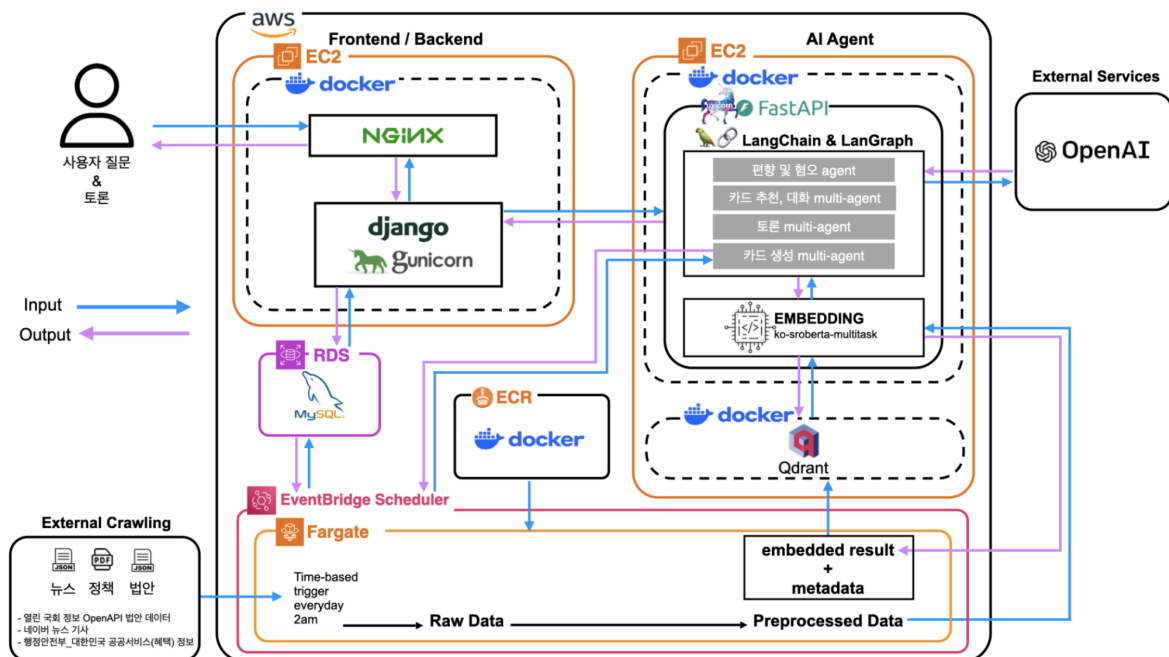


Policy 시스템 구성도

산출물 단계	모델 배포
평가 산출물	시스템 구성도
팀명 / 프로젝트	SKN24 / Policy — 청년을 위한 정책·뉴스 AI 해설 서비스
GitHub	https://github.com/SKNETWORKS-FAMILY-AICAMP/SKN24-FINAL
작성 팀원	김은우, 김현수, 나혜린, 박정은, 전윤우

1. 시스템 아키텍처



2. 서버별 시스템 구성도

① Data Pipeline (AWS Fargate, Python 3.11)

```
# data_pipeline 폴더 구조

data_pipeline/

├─ main.py                # 파이프라인 진입점 (run_news / run_policy 6단계
실행)
├─ config.py              # API 키, DB 접속 정보, 경로 설정
├─ lifecycle_manager.py   # AWS Fargate 태스크 생명주기 관리
├─ requirements.txt
├─ Dockerfile
|
├─ .github/workflows/
|   └─ deploy.yml         # GitHub Actions CI/CD (ECR 빌드 → Fargate 배포)
|
├─ collectors/            # 원본 데이터 수집
|   └─ models.py          # 수집 데이터 공통 모델
|   └─ utils.py
|   └─ news/
|       └─ news_collector.py # Naver 뉴스 API 수집
|       └─ main.py
|   └─ policy/
|       └─ gov24.py        # 공공데이터포털 (Gov24) 정책 수집
|       └─ naver.py        # Naver 정책 관련 뉴스 수집
|       └─ law_collector.py # 법제처 관련 법령 수집
|
├─ parsers/              # 원본 데이터 전처리/파싱
|   └─ news_parser.py
```

```
|   └─ policy_parser.py
|
|   └─ law_parser.py
|
|   └─ utils.py
|
|
└─ storage/                                # 파싱 데이터 저장 (RDS / Qdrant / S3)
    └─ rds.py                             # RDS 공통 연결
    └─ rds_handler.py                     # 뉴스 RDS 저장
    └─ vector_uploader.py                 # Qdrant 벡터 업로드 공통
    └─ news_qdrant_handler.py             # 뉴스 벡터 저장
    └─ policy_rds_handler.py              # 정책 RDS 저장
    └─ policy_rds_uploader.py
    └─ policy_qdrant_handler.py           # 정책 벡터 저장
    └─ policy_qdrant_uploader.py
|
|
└─ cards/                                # AI 카드 생성
    └─ card_generation.py                 # GPT 기반 카드 콘텐츠 생성
    └─ upload_cards.py                   # 생성된 카드 DB 저장
    └─ test_card_save.py
|
|
└─ data/                                # 로컬 임시 데이터 (Fargate 실행 시 마운트)
    └─ news/
        └─ raw/                          # 수집 원본
        └─ preprocessed/                 # 전처리 결과
        └─ metadata/                     # 키워드 캐시 등
    └─ policy/
        └─ policies/
            └─ raw/
            └─ preprocessed/
```

```
| └─ metadata/ # gov24_seen.csv (중복 방지)
|
| └─ related_laws/
|
|   └─ raw/
|
|   └─ preprocessed/
|
| └─ related_news/
|
|   └─ raw/
|
|   └─ preprocessed/
```

Data Pipeline은 매일 새벽 2시 AWS EventBridge에 의해 Fargate 컨테이너로 단발 실행된다. main.py가 진입점이며 크게 수집(collect) → 전처리(parse) → 적재(upload) → 카드 생성 순으로 동작한다.

수집 단계는 collectors 패키지로 분리되어 있다. news 모듈은 Naver 뉴스 API를 aiohttp로 비동기 호출해 기사를 수집하고, policy 모듈은 Gov24 공공서비스 API에서 청년 정책을 수집한 뒤 관련 법령을 국회 법제처 API로 추가 수집한다.

전처리는 parsers 패키지에서 담당한다. 뉴스 기사는 품질 기준에 따라 필터링·중복 제거되고, 정책 데이터는 카테고리별로 구조화된 JSON으로 변환된다. 법령은 PDF 원문에서 조문 단위로 파싱되어 Markdown 형태로 정제된다.

적재는 storage 패키지가 맡는다. 정형 데이터는 AWS RDS(MySQL)에 저장하고, 임베딩 벡터는 Qdrant에 업로드한다. 원문 파일은 AWS S3에 아카이브된다. 마지막으로 cards 모듈이 OpenAI API를 호출해 뉴스·정책 카드를 생성하고 RDS에 저장한다.

라이브러리	버전	용도
aiohttp	최신	비동기 HTTP 뉴스 수집
requests	최신	동기 HTTP 정책·법령 수집
beautifulsoup4	최신	HTML 파싱
trafilatura	최신	웹 본문 추출
apscheduler	최신	스케줄 실행 (매일 02:00)
pdfplumber	최신	PDF 법령 원문 파싱
kiwipiepy	최신	한국어 형태소 분석
pymysql	최신	MySQL RDS 연결
boto3	최신	AWS S3 업로드
qdrant-client	최신	Qdrant 벡터 적재

openai	최신	카드 생성 (GPT-4o)
python-dotenv	최신	환경변수 로드

② AI Agent 서버 (AWS EC2 / Docker, FastAPI + LangGraph, port 8001)

```
# ai_agent 폴더 구조

ai_agent/
├── main.py                # FastAPI 앱 진입점 (lifespan: 임베딩 모델 로드,
                           Qdrant 초기화, LangGraph 빌드)
├── config.py              # 환경변수 및 모델 설정 (GPT-4o/mini, 임베딩 768d,
                           요청 제한 등)
├── utils.py               # 공통 유틸리티 함수
├── embed_documents.py     # 문서 임베딩 실행 스크립트
├── requirements.txt
├── Dockerfile
├── docker-compose.yml    # ai_agent + Qdrant 서비스 정의
|
├── agents/               # LangGraph 기반 AI 에이전트
|   ├── bias_check.py     # 뉴스 편향성 분석 에이전트
|   ├── hate_detection.py # 혐오 표현 감지 에이전트
|   ├── card_generation/  # 카드 생성 에이전트
|   |   ├── news/         # 뉴스 카드 생성
|   |   └── policy/       # 정책 카드 생성
|   ├── chatbot/          # RAG 기반 챗봇 에이전트 (GPT-4o-mini)
|   └── debate/           # 토론 에이전트 (StateGraph + SQLite 체크포인트 +
                           SSE)
|
├── api/                  # FastAPI 라우터
|   └── cards.py           # 카드 생성 엔드포인트
```

```

├── chatbot.py                # 챗봇 엔드포인트
├── debate.py                 # 토론 엔드포인트 (SSE 스트리밍)
├── embed.py                  # 임베딩 엔드포인트
├
├── db/                       # Qdrant 벡터 DB 연동
├   ├── qdrant_client.py
├   ├── qdrant_connect.py
├   ├── qdrant_upload.py
├   └── retriever.py         # RAG 검색기
├
├── data/
├   ├── crawl/               # 크롤링 원본 데이터
├   └── preprocess/          # 전처리 스크립트
├
├── checkpoints/             # SQLite 토론 체크포인트 저장
├── test/                     # 테스트 코드
└── docs/                     # 문서

```

AI Agent 서버는 FastAPI와 LangGraph로 구성된 AI 추론 서버로, Docker Compose에서 ai_agent 컨테이너와 Qdrant 컨테이너를 함께 실행한다. 서버 시작 시 ko-sroberta-multitask 임베딩 모델(768차원)을 메모리에 로드하고, Qdrant 클라이언트와 OpenAI 클라이언트를 초기화한 뒤 LangGraph 토론 그래프를 빌드한다.

agents 패키지에는 기능별 에이전트가 분리되어 있다. debate 에이전트는 LangGraph StateGraph로 AI vs AI / AI vs User 토론을 구현하며, 세션 상태는 SQLite에 체크포인트로 영속 저장되고 클라이언트에는 SSE 스트리밍으로 전달된다. chatbot 에이전트는 Qdrant에서 관련 문서를 Top-5로 검색(RAG)한 뒤 GPT-4o-mini로 답변을 생성하며 최근 6턴의 대화 이력을 유지한다. card_generation 에이전트는 뉴스·정책 각각의 LangGraph 그래프로 카드를 생성한다. 그 외 bias_check와 hate_detection이 편향성 분류와 혐오 표현 감지를 담당한다.

db 패키지는 Qdrant 클라이언트 싱글톤과 임베딩 retriever를 관리하며, api 패키지는 각 에이전트를 FastAPI 라우터로 노출한다.

라이브러리	버전	용도
fastapi	0.128.8	REST / SSE API 프레임워크
uvicorn	0.39.0	ASGI 서버
langgraph	1.2.4	에이전트 워크플로우 오케스트레이션
langgraph-checkpoint-sqlite	3.1.0	토론 세션 상태 영속 (SQLite)
langchain	1.3.4	LLM 추상화 레이어
langchain-openai	1.2.2	OpenAI LLM·임베딩 래퍼
openai	2.29.0	GPT-4o / GPT-4o-mini API
qdrant-client	1.9.2	Qdrant 벡터 검색 (RAG)
sentence-transformers	2.7.0	ko-sroberta 768d 로컬 임베딩
torch	2.2.2 (CPU)	임베딩 모델 런타임
pydantic	2.12.5	요청·응답 스키마 검증
korcen	1.0.3	한국어 욕설 필터
better-profanity	0.7.0	영문 욕설 필터

③ Backend 서버 (AWS EC2 / Docker, Django + Nginx + Gunicorn, port 80)

```
# backend 폴더 구조

backend/

├─ manage.py
├─ requirements.txt
├─ Dockerfile
├─ docker-compose.yml          # Django + Nginx 서비스 정의
├─ gunicorn.conf.py           # Gunicorn + UvicornWorker 설정
├─
├─ config/                    # Django 프로젝트 설정
│   └─ settings.py            # DB, JWT, INSTALLED_APPS 등 전역 설정
│   └─ urls.py                # 전체 URL 라우팅
│   └─ asgi.py
│   └─ wsgi.py
```

```
|
|
├─ apps/                                # Django 애플리케이션
|
|   ├─ cards/                            # 정책/뉴스 카드
|   |
|   |   ├─ models.py
|   |   ├─ serializers.py
|   |   ├─ views.py
|   |   └─ urls.py
|
|   └─ users/                            # 사용자 인증 (JWT)
|   |
|   |   ├─ models.py
|   |   ├─ serializers.py
|   |   ├─ views.py
|   |   └─ urls.py
|
|   └─ chatbots/                         # 챗봇 프록시 (ai_agent → httpx)
|   |
|   |   ├─ models.py
|   |   ├─ views.py
|   |   └─ urls.py
|
|   └─ debates/                          # 토론 프록시 (ai_agent → SSE)
|   |
|   |   ├─ models.py
|   |   ├─ serializers.py
|   |   ├─ views.py
|   |   ├─ urls.py
|   |   └─ templates/debates/
|   |       └─ test.html
|
|
├─ nginx/
|
|   └─ default.conf                      # Nginx 리버스 프록시 설정 (port 80)
|
|
├─ templates/                           # Django 템플릿 (서버사이드 렌더링)
```



```

|   └─ base.html
|
|   └─ components/
|       └─ header.html
|       └─ footer.html
|       └─ card.html
|   └─ pages/
|       └─ index.html
|       └─ landing.html
|       └─ card_list.html
|       └─ card_chatbot.html
|       └─ debate.html
|       └─ login.html
|       └─ signup.html
|       └─ mypage.html
|       └─ ...
|   └─ buses/
|       └─ bus_detail.html
|
└─ static/
    └─ assets/                # 이미지, 캐릭터, 아이콘 등
        └─ images/
        └─ category_cards/
    └─ css/                   # 페이지별 스타일시트
    └─ js/                    # 클라이언트 스크립트

```

Backend 서버는 Django 5.2와 Django REST Framework로 구성된 API 서버로, Nginx 리버스 프록시 뒤에서 Gunicorn + UvicornWorker로 서빙된다. SSE 스트리밍을 지원하기 위해 WSGI 대신 ASGI 진입점(`asgi.py`)을 사용한다.

기능은 도메인 단위의 앱으로 분리되어 있다. **cards** 앱은 뉴스·정책 카드 목록과 상세를 제공하고, **users** 앱은 회원가입·로그인·프로필·북마크·탈퇴를 처리한다. **chatbots** 앱과 **debates** 앱은 AI Agent 서버를 **httpx**로 호출해 챗봇·토론 기능을 프록시하며, 각 세션과 대화 이력을 **MySQL**에 저장한다.

인증은 **JWT** 방식(**django-rest-framework-simplejwt**)으로 구현되며, **access** 토큰 만료는 1일, **refresh** 토큰 만료는 7일이다. 전체 **API**는 기본적으로 **IsAuthenticated** 권한을 요구한다.

라이브러리	버전	용도
Django	5.2.13	웹 프레임워크 (ORM, Admin, 템플릿)
django-rest-framework	3.17.1	REST API
django-rest-framework-simplejwt	5.5.1	JWT 인증 (access 1일 / refresh 7일)
gunicorn	25.3.0	ASGI 프로덕션 서버
uvicorn	0.46.0	Gunicorn UvicornWorker (SSE 지원)
PyMySQL	1.1.2	MySQL RDS 연결
httpx	0.28.1	AI Agent 서버 비동기 HTTP 호출
python-dotenv	1.2.2	환경변수 로드